

Technical Report: GPU Resident FEP Calculations within GROMACS

Yiqi Chen* and Jian Yang

MetaX Technology Inc, Shanghai, China

E-mail: yiqi.chen@metax-tech.com

In this report, we introduce our implementation of FEP calculations on GPUs within GROMACS. The source code of this implementation is released on Github at <https://github.com/yiqichenshallwetalk/GROMACS-FEP-GPU>.

The Original GROMACS FEP Procedure

GROMACS, short for GRONingen MACHine for Chemical Simulations, is a robust, free and open-source MD simulation engine. It was among the pioneering MD packages to leverage the computational power of GPUs. Since the introduction of version 4.5 in 2013,¹ the developers of GROMACS have ported a large portion of its functionalities to GPU. This transition, coupled with the optimization of core algorithms tailored for GPU execution, including the cluster-based pair searching, Verlet list and cutoffs, has propelled GROMACS to deliver exceptional performance on GPUs.² As of the latest official release, GROMACS 2024.2, the software supports the offloading of a wide range of calculations to GPU, including the pairwise nonbonded (NB) computations, bonded calculations, particle mesh Ewald (PME) for long-range electrostatic evaluations and several integration algorithms.

However, most FEP calculations are not currently supported on GPU in GROMACS.

The FEP calculation procedure within GROMACS operates as follows: Initially, the program reads in FEP parameters from the molecular dynamics parameters (MDP) file, such as the molecules or atoms perturbed, the λ -values for van der Waals (VDW), Coulomb and bonded interactions. Subsequently, a separate pair list for FEP atoms is generated and maintained. When operating on a CPU, FEP forces are computed and reduced post NB calculations. In scenarios involving CPU-GPU heterogeneous computing, NB forces are offloaded to the GPU, while the FEP module remains on the CPU. As illustrated in Fig.1(a), these processes can be executed concurrently.

Implementing FEP Calculations on GPU

Here we discuss the implementation detail of FEP calculations on GPU. To optimize a GPU-resident GROMACS FEP workflow, we have offloaded the computation of FEP forces and energies, including the energy differences at foreign λ states, as well as the integration step, directly onto the GPU. As shown in Fig.1(b), the FEP-on-GPU-enabled GROMACS code is capable of performing the entire MD step within the GPU environment. This GPU-resident process significantly reduces data transfers between the CPU and GPU, which are now confined to MD steps requiring pair list reconstruction or the logging of simulation data, events that generally occur at intervals of every 100 to 250 steps. The optimized code adds an option for the *mdrun* command to control the offloading of FEP calculations to GPU. Like the standard NB, bonded and PME computations, users can specify *-fep gpu* to enable GPU acceleration of FEP.

NB FEP Calculations

In the original GROMACS FEP process, the FEP pair-list is generated within the Pair Search module, as illustrated in Fig.1. This list, which details the atoms involved in FEP calculations, is then sent to the CPU for FEP calculations. Each entry in the FEP pair-list identifies an atom and the indices of its interacting partners. In our optimized GPU-

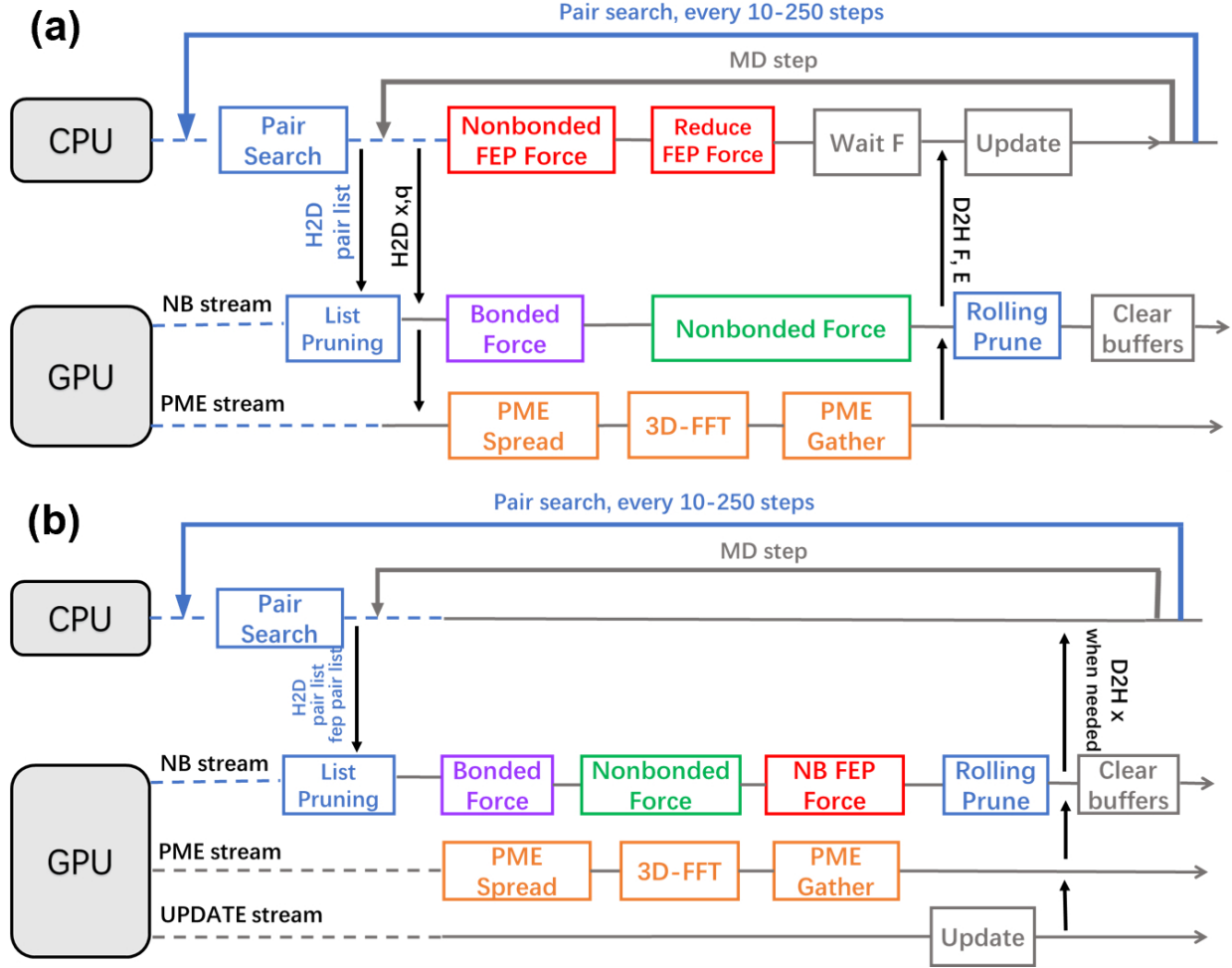


Figure 1: Illustration of one GROMACS FEP MD Step: (a) FEP-on-CPU, 2023.3 official release; (b) FEP-on-GPU, our optimized version. Here "H2D" and "D2H" indicate host-to-device memory copy and device-to-host memory copy, respectively. "x" indicates the 3-dimensional atom coordinates data, "q" is the partial atom charge data, "F" and "E" are the forces and energies computed by GPU kernels.

resident FEP path, the FEP pair-list is transferred to GPU memory, and GPU kernels are launched to calculate both the NB FEP forces and the foreign λ -state energies. As depicted in Fig.1(b), the FEP NB kernel is integrated into the NB stream and launched after the standard NB kernel. This design is predicated in two considerations. First, the NB FEP kernel shares input data with the standard NB kernel, including atom coordinates and forcefield parameters. By placing both kernels within the same GPU stream, we can prevent redundant data movements and enhance memory efficiency. Secondly, the parallel execution of NB and PME kernels already utilizes the majority of GPU computational resources. Therefore, assigning the NB FEP kernel to a separate stream would not yield any additional computational efficiency gains.

Next, we discuss the optimization process of NB FEP kernels on GPU. The NB FEP Lennard-Jones (LJ) potential is calculated as

$$V_{LJ}(r_{ij}) = \frac{(1 - \lambda)C_A^{12} + \lambda C_B^{12}}{r_{ij}^{12}} - \frac{(1 - \lambda)C_A^6 + \lambda C_B^6}{r_{ij}^6}, \quad (1)$$

where λ is the coupling parameter which scales the LJ potential, r_{ij} is the distance between atom i and atom j and C_A and C_B are the LJ parameters of end state A and B, respectively. Note that Eq.1 presents the simple linear interpolation of LJ potential. In practice, a soft core (SC) function is typically applied to eliminate the singularities in the potentials. Currently, our implementation supports the use of the Beutler SC function,³ with parameters configurable in the *mdp* file. For the purpose of discussing the NB FEP GPU algorithm, we use the linear interpolation formula for simplicity. The SC approach follows the same procedure but requires more arithmetic operations.

The most straightforward way of porting these calculations to GPU is by directly casting the CPU computations to a data-parallel algorithm suitable for GPU execution. Algorithm 1 provides the pseudocode for this naive implementation. In this pseudocode, reads from local registers are denoted by $\leftarrow$ while global loads from device memory is denoted by $\Leftarrow$. In this implementation, each GPU thread is assigned to handle one entry in the FEP pair-

list. It begins by loading the coordinates and forcefield parameters of atom i from device memory, as shown on lines 5 through 9 of the pseudocode. Subsequently, the thread loops over j atoms (as indicated by the for-loop starting on line 10) and computes the FEP force f_{ij}^{FEP} depending on the pairwise distance $|d\vec{r}_{ij}|$. Finally, atomic-add operations are utilized to accumulate the NB FEP forces across the threads. For simplicity, this pseudocode does not include the calculations of the perturbed short-ranged Coulomb forces, which can be obtained with the perturbed partial charges.

The naive GPU algorithm, while capable of producing accurate FEP results, falls short in computational efficiency due to several performance-hindering issues. Key challenges include: (1). Severe Thread Divergence: The uneven distribution of interaction counts for each entry in the FEP pair-list, ranging from 1 to 64, results in significant thread divergence. This disparity in workloads within the same warp (a group of threads executed in parallel, fundamental execution unit of GPU) means that some threads complete their tasks more quickly, leading to the underutilization of the GPU’s parallel processing capabilities. (2). Low Utilization: The algorithm employs a number of threads equal to the total entries in the FEP pair-list, which is often not substantial. Therefore, the kernel’s grid size (number of blocks) is small, potentially leading to issues such as low utilization and a long-tail effect, which can lead to a significant performance drop. (3). Memory-Bound Operations: The high latency associated with random, non-coalesced global read operations, denoted by the $\Leftarrow$ symbol in Algorithm 1, renders the kernel memory-bound. This lack of data reuse results in low compute intensity, further impacting the algorithm’s efficiency.

To address the challenges outlined above, we have developed an optimized NB-FEP GPU algorithm. The pseudocode of this algorithm is presented in Algorithm 2. We highlight some key optimizations incorporated into the algorithm: (1). Work Assignment Logic: We transitioned from assigning one thread per entry to assigning one warp per entry. This change yields two significant benefits. Firstly, the workload distribution is more balanced, as computations for each entry are evenly spread across threads within the same warp, thereby

Algorithm 1 Naive Implementation of NB-FEP on GPU

Require: bid is the index of this block

Require: tid is the index of this thread within the block

Require: BS is the number of threads in each block

Require: nri is the number of entries in FEP pair list

```
1:  $tid_{global} \leftarrow bid * BS + tid$  ▷ global thread id
2:  $\vec{F}_i^{sum} \leftarrow \vec{0}$ 
3:  $lambda \leftarrow \text{lambdaList}[stateIdx]$  ▷ coupling parameter  $\lambda$ 
4: if ( $tid_{global} < nri$ ) then
5:    $a_i \leftarrow \text{iinr}[tid_{global}]$  ▷ index of atom  $i$ 
6:    $j_0, j_1 \leftarrow \text{jindex}[tid_{global}], \text{jindex}[tid_{global} + 1]$  ▷ the start and end index of  $j$  atom for this entry
7:    $\vec{x}_i \leftarrow \vec{x}[a_i]$  ▷ coordinates of atom  $i$ 
8:    $C6A_i, C6B_i \leftarrow C6A[a_i], C6B[a_i]$  ▷ the C6 parameter of atom  $i$  at end state A and B
9:    $C12A_i, C12B_i \leftarrow C12A[a_i], C12B[a_i]$  ▷ the C12 parameter of atom  $i$  at end state A and B
10:  for ( $j = j_0$  to  $j_1 - 1$ ) do
11:     $\vec{F}_j^{sum} \leftarrow \vec{0}$ 
12:     $a_j \leftarrow \text{jjnr}[j]$ 
13:     $\vec{x}_j \leftarrow \vec{x}[a_j]$  ▷ coordinates of atom  $j$ 
14:     $C6A_j, C6B_j \leftarrow C6A[a_j], C6B[a_j]$  ▷ the C6 parameter of atom  $j$  at end state A and B
15:     $C12A_j, C12B_j \leftarrow C12A[a_j], C12B[a_j]$  ▷ the C12 parameter of atom  $j$  at end state A and B
16:     $C6A_{ij}, C6B_{ij} \leftarrow C6A_i * C6A_j, C6B_i * C6B_j$ 
17:     $C12A_{ij}, C12B_{ij} \leftarrow C12A_i * C12A_j, C12B_i * C12B_j$ 
18:     $d\vec{r}_{ij} \leftarrow \text{minimum image of } \vec{x}_j - \vec{x}_i$ 
19:    if ( $|d\vec{r}_{ij}| \leq r_{cut}$ ) then
20:       $f_{ij}^{FEP} \leftarrow \text{fepforce}(\vec{r}_{ij}, lambda)$  ▷ compute FEP force at the current  $\lambda$  state
21:       $\vec{F}_j^{sum} \leftarrow \vec{F}_j^{sum} + f_{ij}^{FEP}$ 
22:       $\vec{F}_i^{sum} \leftarrow \vec{F}_i^{sum} + f_{ij}^{FEP}$ 
23:    end if
24:     $\text{atomicAdd}(\vec{F}[a_j], -\vec{F}_j^{sum})$ 
25:  end for
26: end if
27:  $\text{atomicAdd}(\vec{F}[a_i], \vec{F}_i^{sum})$ 
```

enhancing the parallelism of the NB-FEP kernel. Secondly, GPU utilization is improved due to the requirement for a larger grid of threads. (2). Warp Shuffle Operations: We employed efficient warp-level primitives to perform broadcast, data exchange, and reduction operations within a warp, leveraging the capabilities of warp shuffle operations. (3). Memory Layout of Forcefield Parameters: We altered the memory layout of forcefield parameters from a Structure of Arrays (SOA) to an Array of Structures (AOS). Profiling of the NB-FEP kernel indicated a memory-bound characteristic, with the primary bottleneck being non-coalesced global read operations. FEP calculations require forcefield parameters at both end states, leading to a common global read pattern where load operations for several arrays at the same index are executed consecutively (line 8, 9 and 14, 15 in Algorithm 1). In the original SOA layout, values in different arrays are not stored adjacently in memory, and the global load instructions are not coalesced. This was optimized by rearranging the LJ parameter layout to AOS, converting the four arrays of float into one array of float4 (a structure holding 4 floats), as shown on line 10 and 17 in Algorithm 2. This modification makes global load operations more efficient. Overall, the optimized NB-FEP kernel demonstrates a performance improvement of approximately 50% compared to the previous version.

Bonded FEP Calculations

In the original code, FEP-related bonded and restraint forces were categorized as a distinct interaction type, similar to other listed forces that are not supported on the GPU and thus must be computed on the CPU. Recognizing that the computational process for these forces closely mirrors that of the standard bonded forces, except for the scaling of a λ_{bonded} parameter, we fused these calculations into the existing bonded GPU kernel. This approach avoids the need for separate GPU kernel launches. As shown in Fig.1(b), the bonded forces kernel now handles both the non-perturbed and perturbed listed interactions, streamlining the workflow and enhancing efficiency. In addition to adding FEP capabilities to the eight listed force types on the GPU, we also ported six additional types relevant to this study to

Algorithm 2 Optimized Implementation of NB-FEP on GPU

Require: bid is the index of this block

Require: tid is the index of this thread within the block

Require: BS is the number of threads in each block

Require: $warpSize$ is the number of threads within a warp: 32 for Nvidia GPU, 64 for MetaX GPU

Require: nri is the number of entries in FEP pair-list

```
1:  $tid_{global} \leftarrow bid * BS + tid$  ▷ global thread id
2:  $wid_{global} \leftarrow tid_{global} / warpSize$  ▷ global warp id
3:  $wtid \leftarrow tid \% warpSize$  ▷ thread id within warp
4:  $\vec{F}_i^{sum} \leftarrow \vec{0}$ 
5:  $lambda \leftarrow lambdaList[stateIdx]$  ▷ coupling parameter  $\lambda$ 
6: if ( $wid_{global} < nri$ ) then
7:    $a_i \leftarrow iinr[wid_{global}]$  ▷ index of atom i, warp broadcast
8:    $j_0, j_1 \leftarrow jindex[wid_{global}], jindex[wid_{global} + 1]$  ▷ the start and end index of j atom for this entry
9:    $\vec{x}_i \leftarrow \vec{x}[a_i]$  ▷ coordinates of atom i, warp broadcast
10:   $\vec{C}_i \leftarrow \vec{C}[a_i]$  ▷ the LJ parameters of atom i at end states, global read of float4
11:   $C6A_i, C6B_i, C12A_i, C12B_i \leftarrow \vec{C}_i.x, \vec{C}_i.y, \vec{C}_i.z, \vec{C}_i.w$ 
12:  for ( $i = j_0$  to  $j_1$  at interval of  $warpSize$ ) do
13:     $j = i + wtid$ 
14:     $\vec{F}_j^{sum} \leftarrow \vec{0}$ 
15:     $a_j \leftarrow jnr[j]$ 
16:     $\vec{x}_j \leftarrow \vec{x}[a_j]$  ▷ coordinates of atom j
17:     $\vec{C}_j \leftarrow \vec{C}[a_j]$  ▷ the LJ parameters of atom j at end states, global read of float4
18:     $C6A_j, C6B_j, C12A_j, C12B_j \leftarrow \vec{C}_j.x, \vec{C}_j.y, \vec{C}_j.z, \vec{C}_j.w$ 
19:     $C6A_{ij}, C6B_{ij} \leftarrow C6A_i * C6A_j, C6B_i * C6B_j$ 
20:     $C12A_{ij}, C12B_{ij} \leftarrow C12A_i * C12A_j, C12B_i * C12B_j$ 
21:     $d\vec{r}_{ij} \leftarrow$  minimum image of  $\vec{x}_j - \vec{x}_i$ 
22:    if ( $|d\vec{r}_{ij}| \leq r_{cut}$ ) then
23:       $f_{ij}^{FEP} \leftarrow fepforce(\vec{r}_{ij}, lambda)$  ▷ compute FEP force at the current  $\lambda$  state
24:       $\vec{F}_j^{sum} \leftarrow \vec{F}_j^{sum} + f_{ij}^{FEP}$ 
25:       $\vec{F}_i^{sum} \leftarrow \vec{F}_i^{sum} + f_{ij}^{FEP}$ 
26:    end if
27:    atomicAdd( $\vec{F}[a_j], -\vec{F}_j^{sum}$ )
28:  end for
29: end if
30: warpReduce( $\vec{F}_i^{sum}$ ) ▷ use warp shuffle ops to reduce the total force
31: if ( $wtid == 0$ ) then
32:   atomicAdd( $\vec{F}[a_i], \vec{F}_i^{sum}$ )
33: end if
```

the GPU, including bond, angle, and dihedral restraint computations. Currently, 14 most commonly used listed forces out of the 94 types are fully supported on GPU. The details of these listed forces types are listed in 1. One should expect a performance loss if a CPU-only listed force is selected in the FEP simulation, which is due to the data transfer between the CPU and GPU at each step.

Table 1: Listed Force Types Currently Supported on GPUs in This Implementation.

Listed Force Type	Description
F_BOND	Harmonic Bonds Potential
F_HARMONIC	Harmonic Bonds Potential
F_RESTRBONDS	Restarints on Bonds
F_ANGLES	Harmonic Angles Potential
F_UREY_BRADLEY	Urey-Bradley Potential
F_PDIHS	Periodic Proper Dihedrals
F_RBDIHS	Ryckaert-Bellemans Proper Dihedrals
F_IDIHS	Harmonic Improper Dihedrals
F_PIDIHS	Periodic Improper Dihedrals
F_LJ14	Lennard-Jones 1-4 Interactions
F_LJC14_Q	LJ and Coulomb 1-4 Interactions
F_LJC_PAIRS_NB	Nonbonded Pair Interactions
F_ANGRES	Restraints on Angles
F_DIHRES	Restraints on Dihedrals

Force Integration

The capability of offloading PME calculations for Coulomb FEP to GPU was introduced with the GROMACS 2022.1 official release. Therefore, the only task left on CPU within the FEP MD step is the update and constraints module, which is responsible for integrating forces and advancing the system to the subsequent step. Although various update algorithms have been migrated to GPU, the widely employed leap-frog stochastic dynamics (SD) integrator for FEP calculations has yet to be supported on GPU in the latest release. We noticed a noteworthy contribution by Lundborg *et.al* (<https://gitlab.com/gromacs/gromacs/-/tree/gpu-update-sd>), which successfully implemented the SD integration using CUDA. After validating the accuracy of this GPU-enabled SD update across several test cases, we incor-

porated it into our optimized FEP-on-GPU codebase. This enhancement allows all computationally intensive aspects of FEP simulations to be offloaded to GPU, thereby eliminating data transfers and significantly boosting performance. The GPU-resident FEP simulation can be launched by the following command line options: `-nb gpu -pme gpu -bonded gpu -fep gpu -update gpu`. It is important to note that certain update settings remain unsupported on GPU, including: (1). Integrator types other than MD and SD. (2). The Nose-Hoover thermostat for temperature coupling. (3). SD integrator combined with Parrinello-Rahman barostat. For the last case, we recommend using the C-rescale pressure coupling method as an alternative.

Simulation Details

To validate the numerical accuracy and reliability of our GPU-accelerated GROMACS FEP calculations, we simulated four complex systems (4OGI, 4MR3, 4J0R, 3SVG)⁴ previously tested by Aldeghi *et.al.*⁵ using both the original FEP-on-CPU implementation and the optimized FEP-on-GPU implementation. To assess the flexibility of our optimized code, these simulations were conducted on two types of GPU hardware: the NVIDIA A100 GPU and our own product, the MetaX C500 GPU. Following the exact simulation procedure outlined in the literature,⁵ we employed 31 and 42 λ windows for ligand solvent and complex simulations, respectively. For each λ state, a series of equilibration runs were first carried out to achieve system equilibrium. This set consists of a 10,000 energy minimization steps, a 0.5 ns position-restrained canonical (NVT) ensemble simulation and a 1 ns position-restrained isothermal-isobaric (NPT) ensemble simulation. Subsequently, 10 ns unrestrained production runs were performed to gather configurational energies at all λ states. All simulations were conducted at a temperature of 298.15 K, which was coupled using Langevin dynamics.^{6,7} A minor modification to the original setup involved switching the Parrinello–Rahman pressure coupling scheme⁸ to the new C-rescale scheme,⁹ which is compatible with update

operations on GPU. Both barostats effectively control the total pressure during NPT simulations. The same evaluations for the position restraint free energies, long-range dispersion correction and finite-size electrostatic correction were performed to generate the final binding free energies. Three independent runs were performed for each system. The first 1 ns of each production run was discarded as extra equilibration time and samples were decorrelated. The uncorrelated data was then analyzed by the MBAR approach via the GPU-enabled fastMBAR python package.¹⁰ 200 bootstrap sets were constructed from the original dataset to estimate for the statistical uncertainties.

To validate our FEP-on-GPU implementation, the calculated binding free energies were compared against both the previous computational results and the experimental data. The correlations between the calculated and experimental binding affinities were measured by average unsigned error (AUE), root mean square error (RMSE), along with Spearman and Kendall correlation metrics, with standard deviations generated via 1,000 bootstrap resampling sets.

To assess the enhanced performance of our GPU-optimized FEP calculations, performance runs for the four benchmark systems were conducted in isolated environments. The performance of the simulations was evaluated by comparing two sets of codes (the original FEP-on-CPU and the optimized FEP-on-GPU) across three distinct hardware configurations: (1) a 32-core CPU (AMD EPYC 7773X processor @ 1.5 GHz), (2) an NVIDIA 40G A100 GPU, and (3) a MetaX C500 GPU. All simulation and data analysis workflows were managed by the MetaMol FEP workflow management toolkit. The python scripts, input parameter file and all input data files are available online at <https://github.com/yiqichenshallwetalk/FEP-on-GPU-Workflow>.

Validations

Unit Tests

We first attempt to validate our optimized GPU-accelerated FEP calculations using the built-in unit tests provided by GROMACS. The FEP unit test system consists of an acetamide molecule solvated in a water box, totaling 177 atoms. A number of FEP configurations are evaluated within the test suit, including the sequential coupling of VDW and Coulomb interactions, the simultaneous coupling of both forces, the intramolecular coupling, etc. The accuracy of the FEP calculations is assessed by comparing the computed potential energies, forces, and the derivatives of the potential with respect to λ at different steps against the reference data. The default relative tolerances for these comparisons were set at 1.0e-4 for single precision and 1.0e-5 for double precision.

In addition to the standard CPU-based FEP computations, we introduced a new test named "MdrunFEPonGPUPTests" into the unit testing framework. This test evaluates the accuracy of the aforementioned acetamide-water FEP simulations on GPUs. To run this test, one must first compile the test target (e.g., by executing *make tests*) and then initiate the test execution (e.g., with *make test*). The FEP-on-GPU tests successfully passed on both the NVIDIA A100 and MXC500 GPU platforms, confirming the reliability and accuracy of our GPU-enabled FEP calculations.

Numerical Precision of the Configurational Thermodynamic Properties

To assess the numerical precision of FEP calculations on the GPU, we conducted a comprehensive test on a series of configurations collected during FEP simulations. For each benchmark system at every λ state, we chose 7 frames for testing. The properties evaluated in this test include key FEP-relevant energy terms such as Lennard-Jones energy, Coulomb energy, bonded energies, restraints, and dVdl (derivative of potential with respect to λ)

terms. All results were calculated in single precision and compared against CPU-computed values. The test results for several λ states of the ligand-solvent and ligand-protein systems are illustrated in Fig.2 and Fig.3, respectively. For the ligand-solvent system, the relative errors of all energy terms tested are within the tolerance of 1.0×10^{-5} . Due to the unordered nature of atomic-add operations used in GPU, the accumulated energy values exhibit some fluctuation from rounding errors. Consequently, these GPU-computed values should be regarded as accurate.

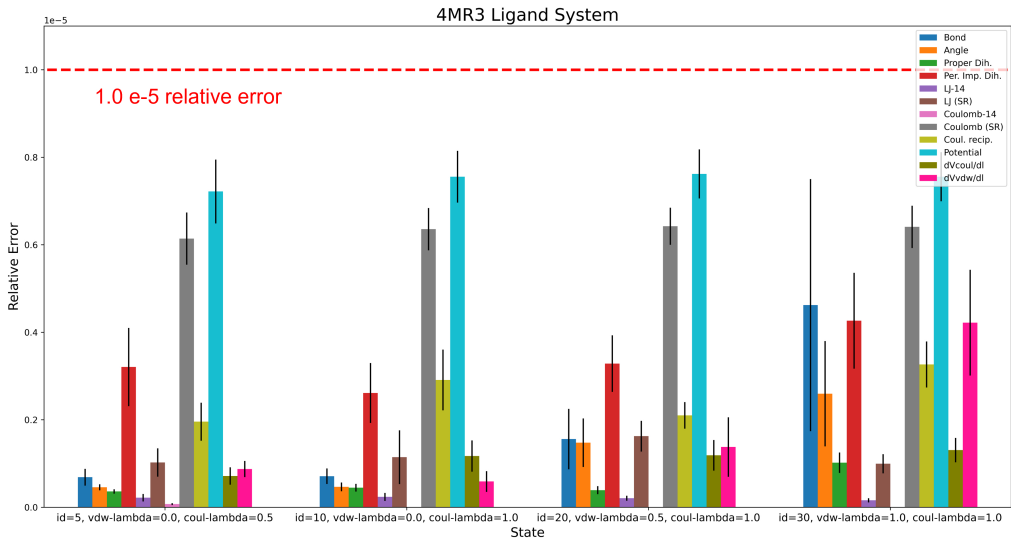


Figure 2: Average relative errors of key FEP energy terms computed on GPU for the 4MR3 ligand-solvent system. Results at four λ states are displayed in the figure, representing different stages along the free energy perturbation path. The dashed red line represents a 1.0×10^{-5} relative error threshold. The energy terms include Bond, Angle, Proper and Improper Dihedrals, LJ-14, LJ short-range, Coulomb-14, Coulomb short-range, Coulomb Reciprocal, total potential energy, dVdwl-VDW and dVdwl-Coulomb.

However, for the ligand-protein configurations, we observe higher relative errors, particularly in the restraint terms. As shown in Fig.3, several relative errors in restraint terms exceed 2.0×10^{-5} . A thorough examination of results across all λ states reveals that the angle restraint term at certain states can exceed 1.0×10^{-4} , which is the GROMACS default relative tolerance of single precision. Upon further investigation, we attribute the high relative errors

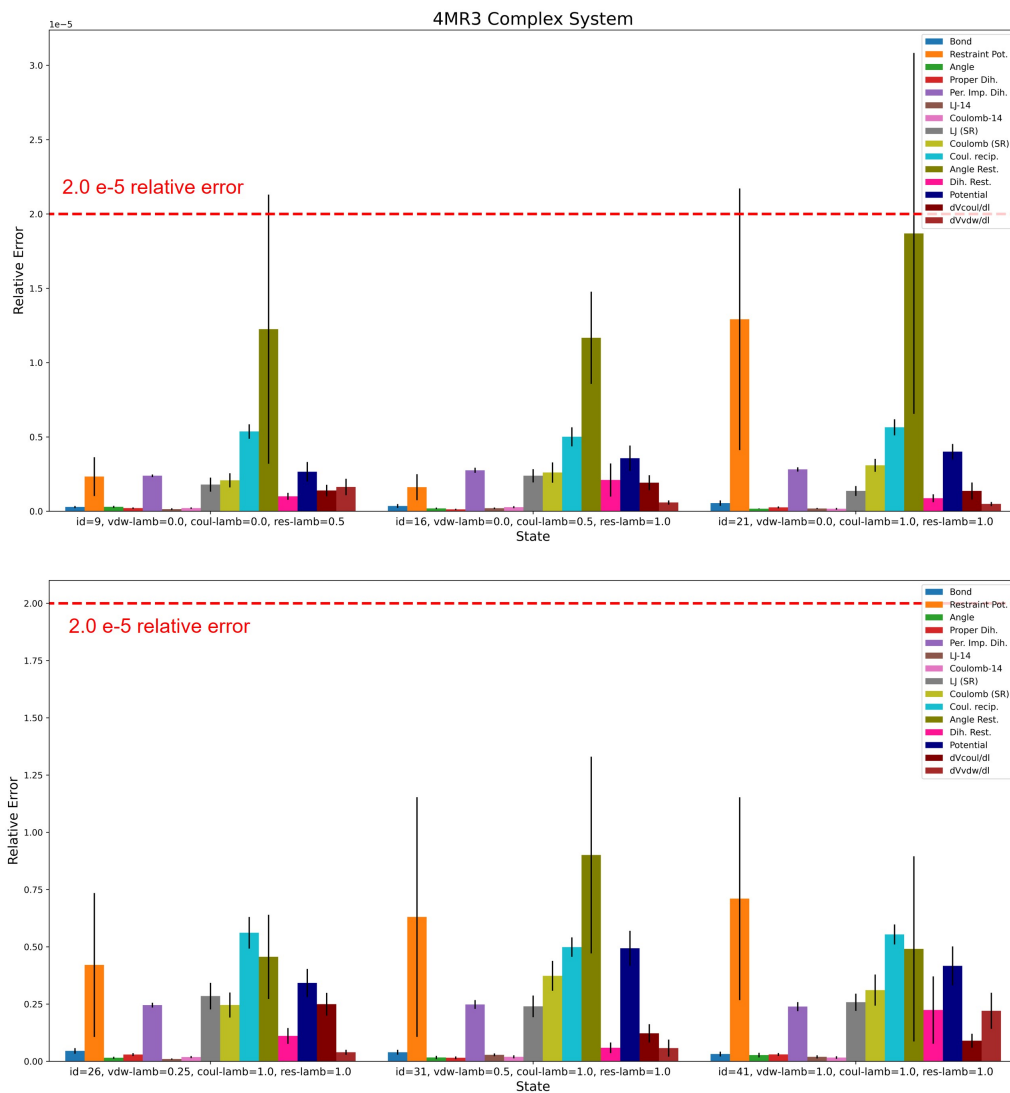


Figure 3: Average relative errors of key FEP energy terms computed on GPU for the 4MR3 ligand-protein system. Results at six λ states are displayed in the figure, representing different stages along the free energy perturbation path. The dashed red line represents a $2.0\text{e-}5$ relative error threshold. The energy terms include Bond, Bond Restraint, Angle, Proper and Improper Dihedrals, LJ-14, LJ short-range, Coulomb-14, Coulomb short-range, Coulomb Reciprocal, Angle Restraint, Dihedral Restraint, total potential energy, dVdl-VDW and dVdl-Coulomb.

to the loss of precision for small values. Angle restraint energies are typically quite small in magnitude, and values close to zero can suffer significant precision loss due to intermediate rounding. Table.2 lists all occurrences of angle restraint values with relative errors higher than 1.0e-4. It is evident that all reference values are close to zero, and the absolute errors are on the order of 1.0e-6 kJ/mol, which has a negligible effect on the total potential energy of the system. Similar results were obtained in the other three benchmark systems. In conclusion, the configurational thermodynamics properties computed on GPU are numerically accurate.

Table 2: Angle Restraints with Large Relative Error. Entry ID is formatted as a combination of the λ state index on the left and the frame index on the right. All energies are in kJ/mol.

ID	CPU Value	GPU Value	Absolute Error	Relative Error
3-0	0.001181466	0.001181341	1.24797e-7	1.05629e-4
5-1	0.001017993	0.001017744	2.49362e-7	2.44954e-4
7-1	3.5412788e-5	3.641033e-5	9.97545e-7	2.81691e-2
8-1	0.005460552	0.005458811	1.74576e-6	3.19705e-4
11-0	0.027801532	0.027796546	4.98630e-6	1.79353e-4
12-2	0.029761706	0.029756719	4.98630e-6	1.67541e-4
13-0	0.018334847	0.018329859	4.98816e-6	2.72059e-4
13-1	0.007411747	0.007401771	9.97586e-6	1.34595e-3
13-6	0.024641814	0.024639322	2.49221e-6	1.01138e-4
14-0	0.012204942	0.012199955	4.98723e-6	4.08624e-4
14-5	0.003915357	0.003910370	4.98723e-6	1.27376e-3
19-0	0.023447257	0.023444763	2.49408e-6	1.06370e-4
28-0	0.027140660	0.027135674	4.98630e-6	1.83721e-4
35-2	0.068381593	0.068389073	7.48038e-6	1.09392e-4
39-0	0.002598600	0.002596106	2.49385e-6	9.59689e-4

End-To-End FEP Workflow Tests

We continue the validation of our FEP-on-GPU GROMACS implementation by conducting comprehensive end-to-end FEP workflow tests on four benchmark systems: 4OGI, 4MR3, 4J0R and 3SVG. These systems have previously been studied computationally and experimentally, providing a solid foundation for comparative analysis. The FEP calculations were executed on both Nvidia A100 and MetaX C500 GPUs. For clarity in the subsequent dis-

cussion, we introduce the following notations to distinguish between various code paths and hardware configurations. **A100-FEP-CPU**: This notation refers to the scenario where non-perturbed NB, PME and bonded computations are offloaded to the A100 GPU, while FEP and update operations remain on CPU (refer to Fig.1(a)). **A100-FEP-GPU**: This label signifies a fully GPU-resident FEP simulation executed on the A100 GPU (see Fig.1(b)). Similarly, **MXC500-FEP-CPU**: This term denotes FEP simulations where the FEP and system update operations are performed on the CPU, utilizing the MetaX C500 GPU for other computations. **MXC500-FEP-GPU**: This notation indicates FEP simulations where all operations are conducted on the MetaX C500 GPU. This systematic approach ensures a thorough evaluation of our FEP-on-GPU implementation across different hardware environments.

The FEP results for the 4MR3 system computed by the various hardware configurations are depicted in Fig.4. Several key thermodynamic properties extracted from the FEP simulations are compared. (1). Potential energy variations during FEP simulations are showcased in subplot (a), highlighting the energy profile of the protein-ligand complex at the 20th λ -state. It is evident that the potential energies computed by all four hardware configurations fluctuate around a common baseline with similar amplitudes, indicating good statistical consistency. (2). The free energy difference between adjacent λ states for both the ligand solvent (subplot (b)) and complex (subplot (c)) systems are presented. The f_i values were computed using the GPU-enabled python library fastMBAR.¹⁰ To ensure the reliability of the results, the initial 1 ns of production run was excluded and the simulation data was decorrelated prior to the MBAR analysis. The comparisons across different configurations for all benchmark systems demonstrate excellent concordance, with all free energy differences aligning within the statistical uncertainty. (3). The free energy differences between the end states for the ligand solvent and ligand-protein complex systems, denoted as ΔG^{solv} and $\Delta G^{complex}$, are derived by the accumulation of f_i values at intermediate states and are depicted in subplots (d) and (e), respectively. These results are benchmarked against

literature values obtained on CPU, serving as a reference standard. For the ligand solvent systems, the ΔG^{solv} values computed from different hardware configurations are statistically consistent, despite varying magnitudes. In the case of ligand-protein complex systems, larger uncertainties and value discrepancies are anticipated due to their greater atomic complexity and sampling challenges. Nonetheless, the overall consistency remains relatively good.

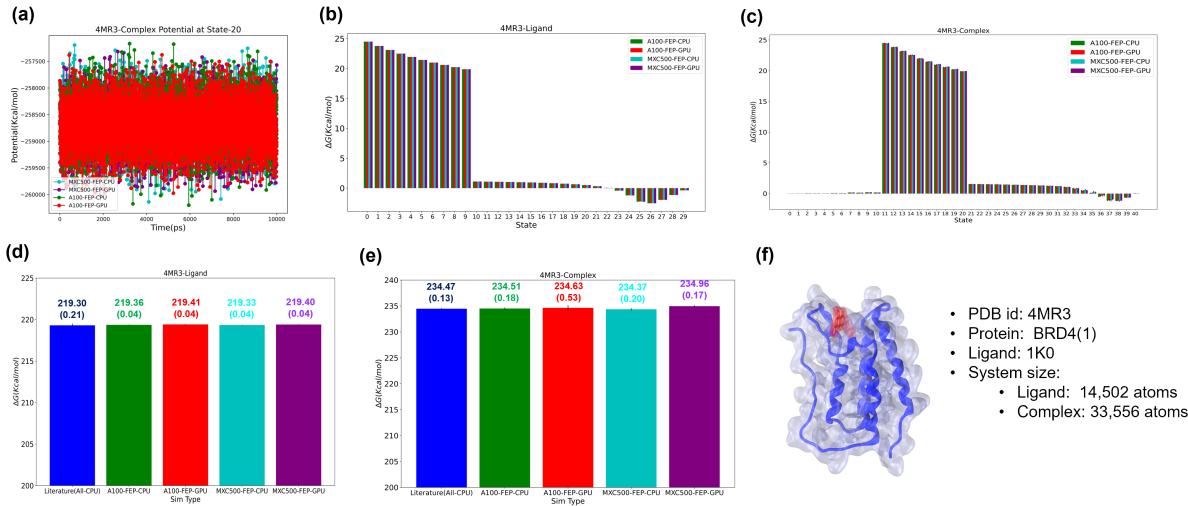


Figure 4: Comparison of FEP results computed by four configurations for 4MR3 protein-ligand system. The colors blue, green, red, cyan and purple represent Literature (All-CPU), A100-FEP-CPU, A100-FEP-GPU, MXC500-FEP-CPU and MXC500-FEP-GPU, respectively. (a) Comparison of the variation of potential energy for the protein-ligand complex system at λ state 20. (b) Free-energy differences at all λ states for the ligand solvent system. (c) Free-energy differences at all λ states for the protein-ligand complex system. (d) Free energy difference between two end states (fully-coupled and uncoupled) for ligand solvent system. (e) Free energy difference between two end states (fully-coupled and uncoupled) for ligand-protein complex system. (f) The system information and crystal structure of 4MR3 with a bound ligand highlighted in red.

The free energy differences for both the ligand solvent and ligand-protein complex systems are then combined to yield the final binding free energy values for the benchmark systems. The necessary free energy corrections, including the long-range dispersion correction, the electrostatic finite-size correlation and the position restriction costs, are sourced from literature. The results are summarized in Table.3, with the values in parentheses representing deviations from experimental results. Notably, all values are within 1.1 kcal/mol from experimental measurements, indicating high prediction accuracy. Fig.5 illustrates the correlations

between the binding free energies calculated using different GPU configurations against experimental (subplot (a)) and literature (subplot (b)) results. Four correlation metrics were evaluated and included in the Figure: AUE, RMSE, Spearman’s and Kendall’s coefficients. It is evident that all points in Fig.5 either lie within or closely align with the 1.0 kcal/mol region, indicating strong agreement between the calculated $\Delta G_{binding}$ values and both the previous experimental and literature results. The AUE and RMSE values are well below 1.0 kcal/mol, while the Spearman’s ρ and Kendall’s τ coefficients exceed 0.9 and 0.8, respectively. These metrics collectively demonstrate that the GPU-generated results exhibit excellent correlation with both experimental and literature data. Table.4 compares the correlation statistics against experimental results across different hardware configurations. The uncertainties in the table are computed through 1,000 bootstrap samples. No significant differences are observed between the various configurations for all four metrics. Consequently, it is concluded the reliability and numerical accuracy of our GPU-enabled GROMACS FEP implementation on both the A100 GPU and the MetaX C500 GPU are on par with the CPU version and closely align with experimental approaches.

Table 3: Summary of FEP binding free energy predictions computed by different configurations. Here the experimental and literature data are from Aldeghi *et.al.*⁵ The values in parentheses are the difference between calculated and experimental results, $\Delta G_{calc} - \Delta G_{exp}$. All energies are in kcal/mol.

System ID	4OGI	4MR3	4J0R	3SVG
Experimental	-9.80 ± 0.10	-9.00 ± 0.10	-8.80 ± 0.10	-7.30 ± 0.00
Literature (All-CPU)	-10.40 ± 0.61	-9.07 ± 0.25	-8.60 ± 0.28	-7.80 ± 0.28
	(-0.60 ± 0.62)	(-0.07 ± 0.27)	(0.20 ± 0.30)	(-0.50 ± 0.28)
A100-FEP-CPU	-10.70 ± 0.50	-9.05 ± 0.18	-8.67 ± 0.16	-7.34 ± 0.14
	(-0.90 ± 0.51)	(-0.05 ± 0.21)	(0.13 ± 0.19)	(-0.04 ± 0.14)
A100-FEP-GPU	-10.98 ± 0.34	-9.12 ± 0.54	-9.01 ± 0.23	-7.26 ± 0.36
	(-1.08 ± 0.35)	(-0.12 ± 0.55)	(-0.21 ± 0.25)	(0.04 ± 0.36)
MXC500-FEP-CPU	-10.74 ± 0.51	-8.93 ± 0.21	-9.37 ± 0.20	-7.97 ± 0.35
	(-0.94 ± 0.52)	(0.07 ± 0.23)	(-0.57 ± 0.22)	(-0.67 ± 0.35)
MXC500-FEP-GPU	-10.83 ± 0.20	-9.46 ± 0.17	-8.86 ± 0.38	-7.81 ± 0.25
	(-1.03 ± 0.22)	(-0.46 ± 0.20)	(-0.06 ± 0.39)	(-0.51 ± 0.25)

Table 4: Statistical properties measuring the correlation between experimental measurements and the binding free energies calculated across all hardware configurations. AUE and RMSE values are in kcal/mol.

Statistics (vs ΔG_{exp})	AUE	RMSE	Spearman ρ	Kendall τ
Overall	0.436 ± 0.095	0.577 ± 0.095	0.905 ± 0.064	0.819 ± 0.078
Literature (All-CPU)	0.440 ± 0.093	0.575 ± 0.098	0.902 ± 0.065	0.818 ± 0.081
A100-FEP-CPU	0.435 ± 0.094	0.576 ± 0.090	0.909 ± 0.059	0.822 ± 0.073
A100-FEP-GPU	0.438 ± 0.101	0.574 ± 0.097	0.906 ± 0.067	0.814 ± 0.083
MXC500-FEP-CPU	0.441 ± 0.101	0.575 ± 0.093	0.906 ± 0.062	0.820 ± 0.073
MXC500-FEP-GPU	0.438 ± 0.099	0.575 ± 0.093	0.905 ± 0.062	0.819 ± 0.072

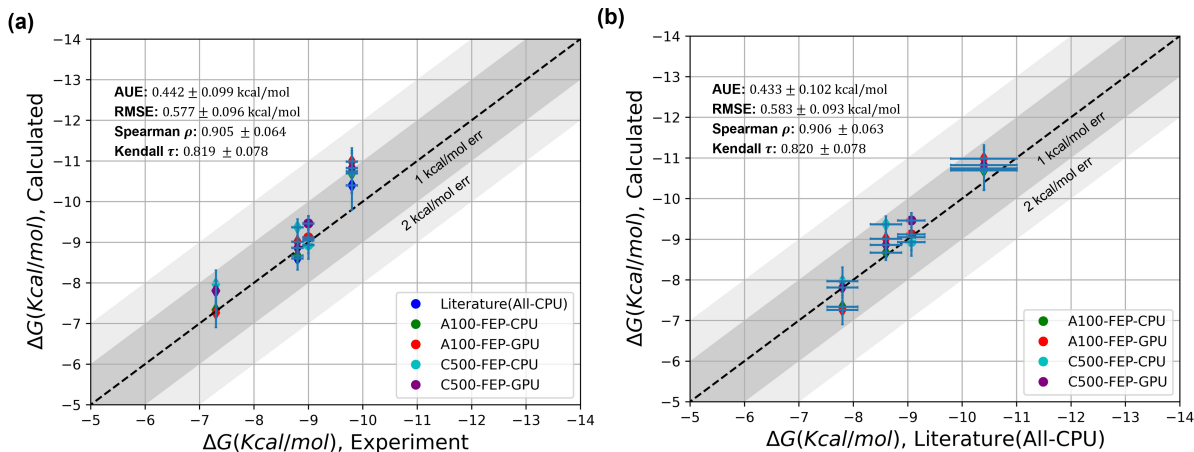


Figure 5: Binding free energy correlation plots for validations. (a) Experimental vs Calculated $\Delta G_{binding}$. The colors blue, green, red, cyan and purple represent Literature (All-CPU), A100-FEP-CPU, A100-FEP-GPU, MXC500-FEP-CPU and MXC500-FEP-GPU, respectively. (b) $\Delta G_{binding}$ calculated on CPU vs GPU. The x-axis contains the values reported in literature which are computed on CPU while the y-axis has values computed on GPU with different configurations. The colors green, red, cyan and purple represent A100-FEP-CPU, A100-FEP-GPU, MXC500-FEP-CPU and MXC500-FEP-GPU, respectively. The dark shaded region spans ± 1.0 kcal/mol; the lighter region spans ± 2.0 kcal/mol.

Performance Analysis

The performance for all four benchmark systems across different hardware configurations are shown in Fig.6. The performance data was measured and collected for FEP production runs in isolated computing environments. The final performance values and error bars in the figure were derived from the average and standard deviations across all λ -windows for both the ligand solvent (31 windows) and the ligand-protein complex (42 windows) systems. Despite variations in total system size and overall performance metrics, the speed-up ratios of GPU-accelerated configurations over CPU remain remarkably consistent across the four test systems.

We first discuss the performance of FEP calculations on the A100 GPU. For both the ligand solvent and ligand-protein complex systems, the average speed-up ratio of the FEP-on-CPU and FEP-on-GPU code paths on A100 compared to the 32-core CPU configuration are approximately 350% and 800%, respectively. The acceleration of the FEP-on-CPU path, which utilizes the official GROMACS release, mainly comes from the offloading of the compute-intensive NB and PME tasks on GPU. Our optimized FEP-on-GPU path enhances overall performance further by dispatching the FEP-related calculations and the update tasks to the GPU, thus minimizing data transfers between the CPU and GPU. This GPU-resident implementation achieves more than a 2-fold increase in speed compared to the original GROMACS FEP calculations. Fig.7 illustrates the timeline profiles collected by the Nvidia Profiler tool for FEP-on-CPU simulations (subplot (a)) and GPU-resident FEP simulations (subplot (b)). It is observed that the kernels in GPU-resident simulations align very closely because there are no excessive CPU-bound and data transfer operations. Conversely, for the FEP-on-CPU simulations, once the calculations of NB and PME forces are completed on the GPU, the results are transferred to the CPU for FEP calculations and update tasks. The updated atom coordinates are sent back to the GPU to proceed with the next MD step. In Fig.7(a), the gaps between MD steps represent the time costs for CPU computations, and the green and pink boxes denote host-to-device and device-to-host data

copy operations, respectively. As a result of these operations, the total time for a single MD step increases from 399 microseconds for the GPU-resident path to 976 microseconds for the FEP-on-CPU path. This ratio is consistent with the overall performance boost of 250% for the FEP-on-GPU implementation.

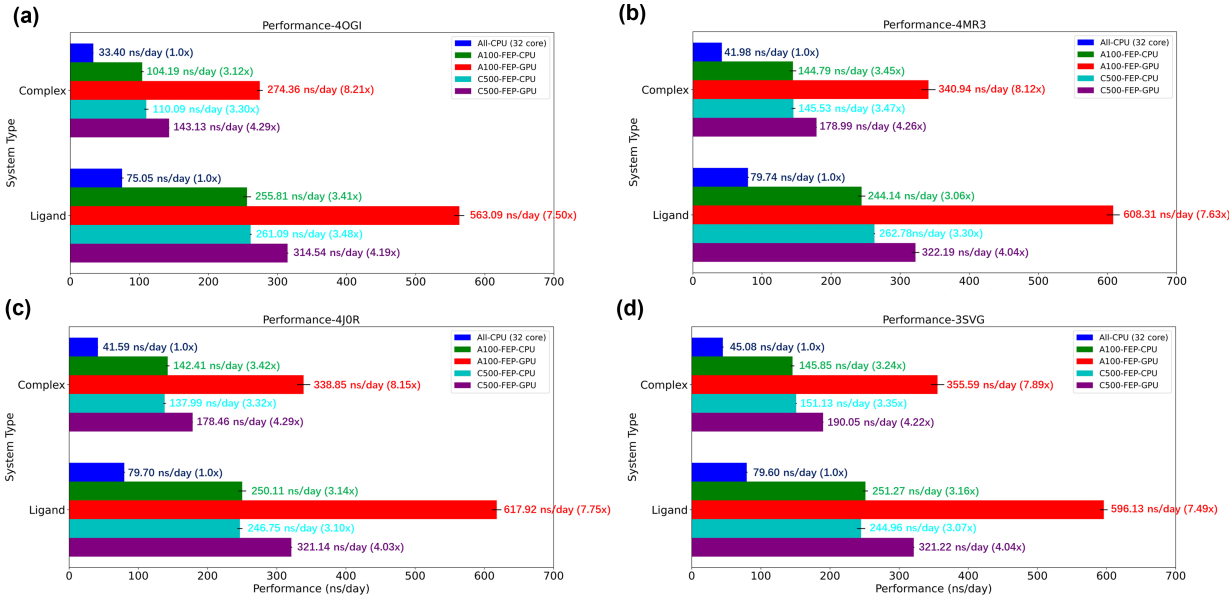


Figure 6: Performance of different configurations for both the ligand and complex systems: (a) 4OGI system, (b) 4MR3 system, (c) 4J0R system, (d) 3SVG system. The colors blue, green, red, cyan and purple represent the 32-core CPU, A100-FEP-CPU, A100-FEP-GPU, MXC500-FEP-CPU and MXC500-FEP-GPU configurations, respectively.

The MXC500 general-purpose GPU, the flagship product of MetaX Technology Inc.,¹¹ supports a diverse range of applications, including AI training and inference operations, as well as scientific computing tasks. The comparison between the MXC500 GPU and the A100 GPU is shown in Table.5. In this study, we conducted GPU-accelerated GROMACS FEP calculations using the MXC500 GPU to assess its performance. As shown in Fig.6, the MXC500 GPU demonstrates a significant performance enhancement, achieving over three-fold and four-fold speed-ups compared to a 32-core CPU configuration for the FEP-on-CPU and FEP-on-GPU code paths, respectively. Although these acceleration results are notably impressive, they still fall short of the performance on the A100 GPU. A deeper examination of the performance discrepancy reveals that the NB and FEP kernels are heavily

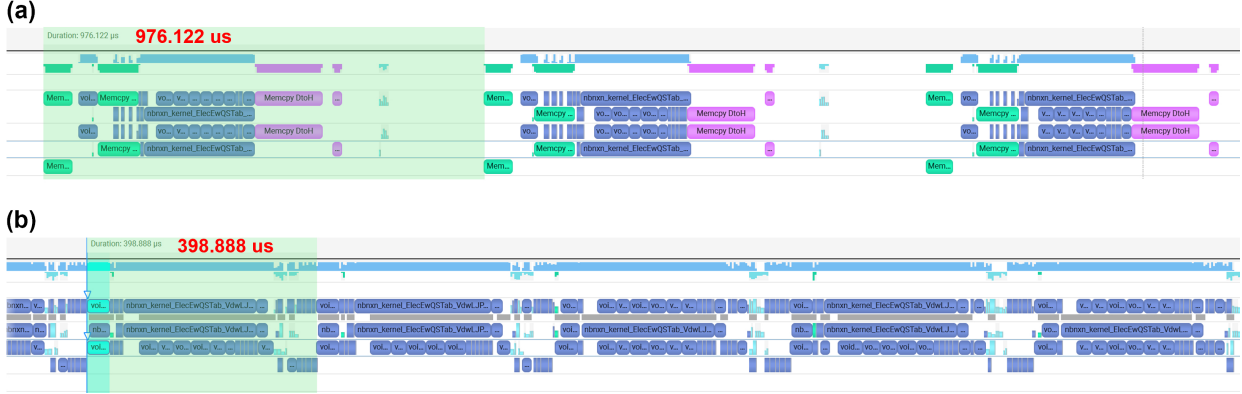


Figure 7: Timeline view profiles for FEP calculations on the A100 GPU. (a) The FEP-on-CPU simulation path; (b) The GPU-resident FEP simulation path. The shaded region represents a single MD step. The bold red text is the total time cost for a single step.

bound by compute-intensive operations. The most frequently executed instructions include the transcendental functions and the fused-multiply-add (FMA) operations. The MXC500 GPU’s hardware architecture necessitates serial execution of transcendental functions, unlike Nvidia GPUs, which can perform these operations asynchronously. This disparity impacts the efficiency of transcendental function instructions. Furthermore, as shown in Table.5, the theoretical peak performance of FP32 (float) on the MXC500 is 15.0 Tflops, approximately 77% of A100’s 19.5 Tflops. In conclusion, the combination of the two factors (the serial execution of transcendental functions and the lower theoretical peak performance) limits the overall performance of the GPU-resident FEP calculations on MXC500 GPU.

Table 5: Specifications of the A100 and MXC500 GPUs.

GPU Name	Vendor	TFLOPS(FP32)	GPU Memory	Bandwidth
A100	NVIDIA	19.5	40 GB	1.56 TB/s
MXC500	METAX	15.0	64 GB	1.84 TB/s

Fig.8 presents a comparative analysis of the total execution time for the end-to-end FEP workflow across various configurations. The end-to-end simulation time is calculated based on the overall workload, which includes 31 independent λ state simulations of the ligand solvent system and 42 complex simulations, and the average simulation speed as shown in Fig.6. The total simulation time of 400 hours on the 32-core CPU configuration

is significantly reduced to roughly 120 hours for both the A100-FEP-CPU and MXC500-FEP-CPU configurations. When employing the optimized FEP-on-GPU code path, the required time is further shortened to approximately 95 hours for the MXC500 GPU and 50 hours for the A100 GPU. These results demonstrate the substantial acceleration potential of GPUs in executing FEP workflows. Moreover, considering the independent nature of FEP simulations, the time-to-solution can be substantially reduced by partitioning λ states across multiple GPUs. Therefore, the GPU-resident GROMACS FEP implementation offers an open-source, accurate, and high-performance solution for the prediction of binding free energies.

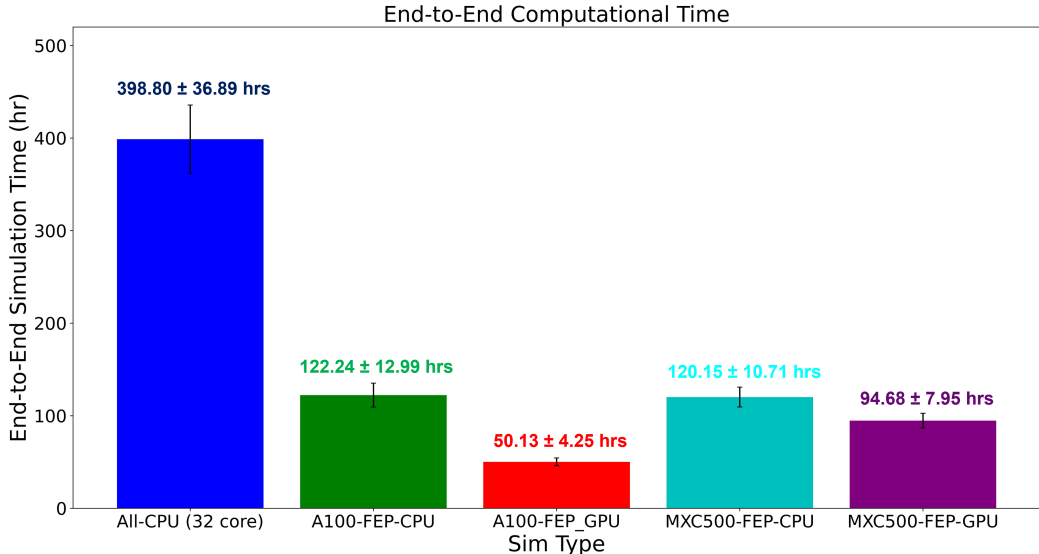


Figure 8: The comparison of the end-to-end simulation time of FEP calculations. The colors blue, green, red, cyan and purple represent the 32-core CPU, A100-FEP-CPU, A100-FEP-GPU, MXC500-FEP-CPU and MXC500-FEP-GPU configurations, respectively.

References

- (1) Pronk, S.; Páll, S.; Schulz, R.; Larsson, P.; Bjelkmar, P.; Apostolov, R.; Shirts, M. R.; Smith, J. C.; Kasson, P. M.; van der Spoel, D.; Hess, B.; Lindahl, E. GROMACS 4.5: a high-throughput and highly parallel open source molecular simulation toolkit. *Bioinformatics* **2013**, *29*, 845–854.
- (2) Páll, S.; Zhmurov, A.; Bauer, P.; Abraham, M.; Lundborg, M.; Gray, A.; Hess, B.; Lindahl, E. Heterogeneous parallelization and acceleration of molecular dynamics simulations in GROMACS. *The Journal of Chemical Physics* **2020**, *153*.
- (3) Beutler, T. C.; Mark, A. E.; van Schaik, R. C.; Gerber, P. R.; van Gunsteren, W. F. Avoiding singularities and numerical instabilities in free energy calculations based on molecular simulations. *Chemical Physics Letters* **1994**, *222*, 529–539.
- (4) Aldeghi, M.; Heifetz, A.; Bodkin, M. J.; Knapp, S.; Biggin, P. C. Aldeghi et al. Files for absolute free energy calculations in gromacs. — zenodo.org. <https://zenodo.org/records/57131>.
- (5) Aldeghi, M.; Heifetz, A.; Bodkin, M. J.; Knapp, S.; Biggin, P. C. Accurate calculation of the absolute free energy of binding for drug molecules. *Chem. Sci.* **2016**, *7*, 207–218.
- (6) Gunsteren, W. F. V.; Berendsen, H. J. C. A Leap-frog Algorithm for Stochastic Dynamics. *Molecular Simulation* **1988**, *1*, 173–185.
- (7) Goga, N.; Rzepiela, A. J.; de Vries, A. H.; Marrink, S. J.; Berendsen, H. J. C. Efficient Algorithms for Langevin and DPD Dynamics. *Journal of Chemical Theory and Computation* **2012**, *8*, 3637–3649, PMID: 26593009.
- (8) Parrinello, M.; Rahman, A. Polymorphic transitions in single crystals: A new molecular dynamics method. *Journal of Applied Physics* **1981**, *52*, 7182–7190.

- (9) Bernetti, M.; Bussi, G. Pressure control using stochastic cell rescaling. *The Journal of Chemical Physics* **2020**, *153*.
- (10) Ding, X.; Vilseck, J. Z.; Brooks, C. L. Fast Solver for Large Scale Multistate Bennett Acceptance Ratio Equations. *Journal of Chemical Theory and Computation* **2019**, *15*, 799–802.
- (11) MetaX metax-tech.com. <https://www.metax-tech.com/en>.